

Pertemuan 2

Fungsi, Subprogram, dan Rekursi Awal

Pemrograman untuk Ilmu Data
dan Komputasi

MA25-21008 Kelas RA dan RB

Triyana Muliawati, S.Si., M.Si.

Rifky Fauzi

Program Studi Matematika
Institut Teknologi Sumatera

Semester Ganjil 2026/2027

Hari ini

- Program mulai dipecah menjadi subprogram.
- Fungsi dipakai untuk memberi nama pada pros
- Rekursi dikenalkan sebagai fungsi yang memanggil dirinya sendiri

Dari M1 ke M2

- M1: ekspresi, variabel, input/output, percabangan, dan pengulangan.
- M2: potongan logika yang sama diberi nama dan dipanggil ulang.
- Program mulai ditulis sebagai kumpulan bagian kecil yang jelas.

Yang ditekankan

- Subprogram, fungsi, parameter, argumen, dan `return`.
- Scope lokal dan global.
- Multiple return, unpacking, fungsi sebagai objek, dan lambda.
- Rekursi dasar dan kondisi berhenti.

Apa itu subprogram?

Definisi kerja

Subprogram adalah bagian program yang diberi nama untuk mengerjakan satu tugas tertentu. Subprogram dapat dipanggil dari bagian lain program tanpa menulis ulang langkah-langkahnya.

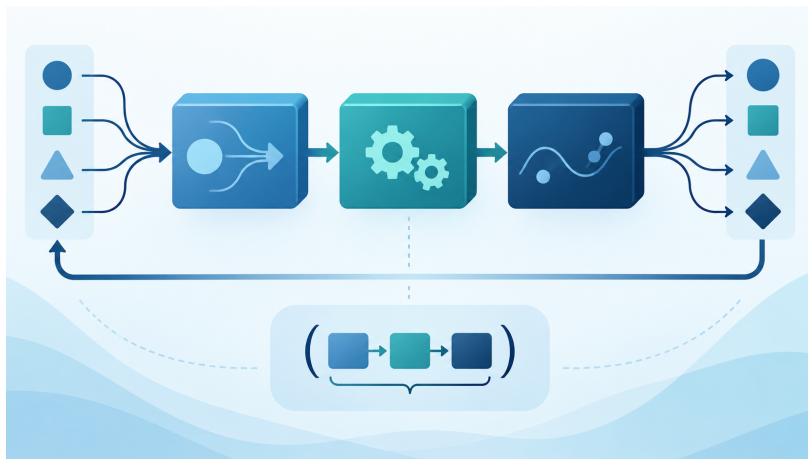
Dalam banyak bahasa

- **Prosedur**: mengerjakan aksi, sering tanpa nilai balik.
- **Fungsi**: mengolah input dan mengembalikan nilai.

Dalam Python

- Keduanya ditulis dengan `def`.
- Jika tidak ada `return`, hasilnya `None`.
- Perbedaan utamanya ada pada tujuan pemakaian.

Makna fungsi dalam program



Cara membacanya

Fungsi adalah unit bernama yang menerima masukan, menjalankan aturan tertentu, lalu

```
def nama_fungsi(parameter_1, parameter_2):  
    langkah_1  
    langkah_2  
    return hasil
```

Saat didefinisikan

- **def**: membuat definisi fungsi.
- **Parameter**: nama masukan di dalam fungsi.
- **Badan fungsi**: langkah-langkah yang dikerjakan.

Saat dipanggil

- **Argumen**: nilai yang dikirim saat pemanggilan.
- **return**: nilai yang dikirim balik.
- Setelah **return**, eksekusi fungsi selesai.

Contoh pertama: fungsi dari rumus

$$p(x) = 2 - 3x + x^2$$

$$p(4) = 2 - 12 + 16 = 6$$

```
def polinom_sederhana(x):  
    return 2 - 3*x + x**2  
  
hasil = polinom_sederhana(4)  
print(hasil)
```

Perhatikan

x pada definisi adalah parameter. Nilai 4 saat pemanggilan adalah argumen. Nilai yang dipakai pada operasi berikutnya berasal dari `return`.

Parameter dan argumen

```
def luas_persegi_panjang(panjang, lebar):  
    return panjang * lebar  
  
A = luas_persegi_panjang(5, 3)  
B = luas_persegi_panjang(lebar=3, panjang=5)
```

Positional argument

Urutan argumen penting. Nilai pertama masuk ke parameter pertama.

Keyword argument

Nama parameter disebut langsung. Urutan argumen menjadi lebih mudah dibaca.

Empat bentuk fungsi: tanpa nilai balik

Tanpa argumen, tanpa return

```
def salam():  
    print("Selamat belajar")
```

Cocok untuk menampilkan pesan atau instruksi.

Dengan argumen, tanpa return

```
def tampil_kuadrat(x):  
    print(x**2)
```

Berguna jika hasil hanya ingin ditampilkan, bukan dihitung lanjut.

Catatan

Jika tidak ada return, Python tetap mengembalikan None.

Empat bentuk fungsi: dengan nilai balik

Tanpa argumen, dengan return

```
def konstanta_pi():  
    return 3.14159
```

Cocok untuk konstanta atau konfigurasi kecil.

Dengan argumen, dengan return

```
def kuadrat(x):  
    return x**2
```

Paling sering dipakai dalam komputasi matematika.

Prinsip

Jika hasil akan dipakai pada perhitungan berikutnya, gunakan return.

print tidak sama dengan return

```
def g(x):  
    return x**2 - 1  
  
def h(x):  
    print(g(x))
```

```
y = h(3)  
print(y)  
  
z = h(3) + 2 # error
```

Yang terjadi

`h(3)` menampilkan hasil `g(3)`, tetapi tidak mengembalikan nilai untuk dipakai pada operasi aritmatika. Karena itu `h(3)+2` gagal.

Fungsi yang mengembalikan lebih dari satu nilai

```
def stat_ringkas(a, b, c, d):  
    nilai = [a, b, c, d]  
    minimum = min(nilai)  
    maksimum = max(nilai)  
    rata = sum(nilai) / len(nilai)  
    rentang = maksimum - minimum  
    return minimum, maksimum, rata, rentang  
  
mn, mx, r, rg = stat_ringkas(4, 7, 1, 9)
```

Apa yang terjadi?

Python mengembalikan satu objek bertipe tuple. Unpacking membagi isi tuple ke beberapa variabel.

Tuple dan unpacking

```
hasil = stat_ringkas(4, 7, 1, 9)
print(hasil)
print(type(hasil))
```

```
minimum, maksimum, rata, rentang = hasil

# jumlah variabel harus cocok
x, y = hasil # ValueError
```

Mengapa penting?

Satu proses komputasi sering menghasilkan beberapa informasi: parameter, galat, jumlah iterasi, atau status konvergensi.

Scope: variabel lokal dan global

```
x = 10

def f(a):
    x = a + 2
    return x

print(f(5))
print(x)
```

Variabel lokal

x di dalam fungsi hanya hidup di dalam fungsi tersebut.

Variabel global

x di luar fungsi tetap bernilai 10. Nilai ini tidak berubah hanya karena ada x lokal.

Mengapa variabel global perlu hati-hati?

```
total = 0

def tambah(x):
    global total
    total = total + x
    return total
```

Masalah yang sering muncul

- Hasil fungsi bergantung pada riwayat pemanggilan.
- Test menjadi tidak mandiri.
- Kesalahan sulit dilacak karena nilai dapat berubah dari tempat lain.

Saran awal

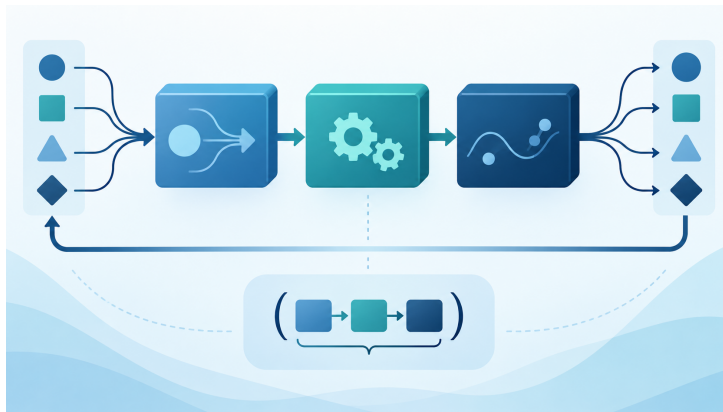
Utamakan fungsi yang menerima input secara eksplisit dan mengembalikan output secara eksplisit.

$$E(p, q) = \frac{1}{1 + d(p, q)^2}$$

$$d(p, q) = \sqrt{(p_1 - q_1)^2 + (p_2 - q_2)^2}$$

```
def jarak(p, q):  
    dx = p[0] - q[0]  
    dy = p[1] - q[1]  
    return (dx**2 + dy**2)**0.5  
  
def energi(d):  
    return 1 / (1 + d**2)  
  
def energi_dua_titik(p, q):  
    return energi(jarak(p, q))
```

Komposisi fungsi



Jika g menerima x dan f menerima hasil dari g , maka $(f \circ g)(x) = f(g(x))$. Dalam Python, ide yang sama muncul saat satu fungsi memanggil fungsi lain.

Fungsi sebagai objek

```
def proses(x, f):  
    return f(x)  
  
def kuadrat(x):  
    return x**2  
  
def tambah1(x):  
    return x + 1  
  
print(proses(5, kuadrat))  
print(proses(5, tambah1))
```

Makna penting

Nama kuadrat tanpa tanda kurung adalah objek fungsi. Objek itu dapat dikirim sebagai argumen ke fungsi lain.

List fungsi dan pipeline

```
def kuadrat(x):  
    return x**2  
  
def tambah1(x):  
    return x + 1  
  
def pipeline(x, ops):  
    hasil = x  
    for f in ops:  
        hasil = f(hasil)  
    return hasil  
  
ops = [abs, kuadrat, tambah1]  
print(pipeline(-3, ops))
```

Baca alurnya

Nilai -3 diproses oleh `abs`, lalu `kuadrat`, lalu `tambah1`. Ini mirip alur transformasi data.

Lambda: fungsi pendek untuk operasi pendek

$$f(x) = 3x^2 - 2x + 1$$

```
f = lambda x: 3*x**2 - 2*x + 1
```

```
xnilai = range(-3, 4)
y = [f(x) for x in xnilai]

z = list(map(lambda x: 3*x**2 - 2*x + 1,
             xnilai))
```

Gunakan secara wajar

Lambda cocok untuk fungsi satu baris yang jelas. Jika logika mulai panjang, fungsi bernama lebih baik.

List comprehension untuk evaluasi fungsi

```
def f(x):  
    return 3*x**2 - 2*x + 1  
  
xnilai = [-3, -2, -1, 0, 1, 2, 3]  
y = [f(x) for x in xnilai]
```

Cara membaca

Untuk setiap x di $xnilai$, hitung $f(x)$, lalu simpan hasilnya ke list baru.

Catatan

Di M2, list dipakai seperlunya agar fungsi bisa diterapkan pada beberapa nilai. Struktur list, dictionary, dan array dibahas lebih kuat pada M3.

Definisi kerja

Rekursi terjadi ketika sebuah fungsi memanggil dirinya sendiri untuk menyelesaikan masalah yang lebih kecil.

Dua bagian wajib

- **Base case:** kondisi berhenti.
- **Recursive case:** memanggil fungsi pada masalah yang lebih kecil.

Pertanyaan saat membaca rekursi

- Apa yang menjadi ukuran masalah?
- Apakah ukuran itu berkurang?
- Kapan pemanggilan berhenti?

Contoh rekursi: faktorial

$$0! = 1, \quad n! = n(n-1)!$$

$$5! = 5 \cdot 4 \cdot 3 \cdot 2 \cdot 1$$

```
def faktorial(n):  
    if n == 0:  
        return 1  
    return n * faktorial(n - 1)  
  
print(faktorial(5))
```

Jejak singkat

`faktorial(5)` memanggil `faktorial(4)`, lalu `faktorial(3)`, sampai mencapai `faktorial(0)`. Setelah base case tercapai, hasil dikalikan kembali ke atas.

Rekursi pada barisan

$$a_0 = 2, \quad a_n = 3a_{n-1} - 2$$

Untuk menghitung a_5 , fungsi perlu mengetahui a_4 , a_3 , dan seterusnya sampai a_0 .

```
def a(n):  
    if n == 0:  
        return 2  
    return 3*a(n - 1) - 2  
  
for k in range(6):  
    print(k, a(k))
```

Catatan

Rekursi cocok ketika definisi masalah memang bergantung pada versi yang lebih kecil dari dirinya sendiri.

Kesalahan umum pada rekursi

Tidak ada base case

```
def turun(n):  
    return turun(n - 1)
```

Fungsi terus memanggil dirinya sendiri.

Tidak menuju base case

```
def salah(n):  
    if n == 0:  
        return 0  
    return salah(n + 1)
```

Untuk $n > 0$, pemanggilan makin jauh dari 0.

Hubungan dengan halt

Sebuah fungsi rekursif harus bergerak menuju kondisi berhenti. Jika tidak, program tidak halt untuk input tertentu.

Merancang fungsi yang mudah diuji

```
def jarak_1d(x, y):  
    return abs(x - y)  
  
assert jarak_1d(3, 3) == 0  
assert jarak_1d(3, 8) == 5  
assert jarak_1d(-2, 3) == 5
```

Kriteria awal

- Input jelas.
- Output jelas.
- Tidak bergantung pada variabel global.
- Dapat diuji dengan contoh yang jawabannya diketahui.

Checklist membaca fungsi

Saat melihat fungsi baru, tanyakan lima hal ini

- ➊ Apa tugas yang dikerjakan fungsi?
- ➋ Apa input yang dibutuhkan?
- ➌ Apa output yang dikembalikan?
- ➍ Apakah fungsi hanya mencetak, atau benar-benar mengembalikan nilai?
- ➎ Apakah fungsi bergantung pada keadaan di luar dirinya?

Untuk mahasiswa Matematika

Biasakan menulis fungsi sebagai padanan komputasional dari objek matematika: fungsi, transformasi, norma, jarak, ringkasan, atau operator.

Materi quiz

- Membaca definisi fungsi dan memprediksi output.
- Membedakan parameter dan argumen.
- Membedakan `print`, `return`, dan `None`.
- Menelusuri scope lokal dan global.
- Menentukan hasil unpacking dan pemanggilan fungsi sebagai argumen.
- Mengidentifikasi base case dan recursive case.

Cara belajar

Kerjakan prediksi dahulu. Setelah itu jalankan Python untuk memeriksa dan menuliskan koreksi pemahaman.

Latihan akhir 1: menulis fungsi

Tulis fungsi dari spesifikasi

Buat fungsi `stat_ringkas(data)` yang menerima list bilangan dan mengembalikan tiga nilai: minimum, maksimum, dan rata-rata. Jangan menggunakan NumPy.

```
data = [4, 7, 1, 9, 3, 6]
mn, mx, rata = stat_ringkas(data)
```

Yang harus dicek

Fungsi harus mengembalikan nilai, bukan hanya mencetak. Uji pada list dengan satu elemen dan list berisi bilangan negatif.

Latihan akhir 2: diagnosa kesalahan

```
def rata(data):  
    total = 0  
    for x in data:  
        total = total + x  
    return total / len(data)
```

Tugas

- 1 Prediksi hasil untuk `data=[2,4,6,8]`.
- 2 Temukan baris yang menyebabkan kesalahan logika.
- 3 Perbaiki fungsi.
- 4 Tambahkan penanganan untuk list kosong.

Latihan akhir 3: buat fungsi rekursif

Barisan rekursif

Diberikan

$$a_0 = 1, \quad a_n = 2a_{n-1} + 1.$$

Buat fungsi `barisan(n)` untuk menghitung a_n .

```
for k in range(6):  
    print(k, barisan(k))
```

Yang ditulis di jawaban

Tandai base case, recursive case, dan alasan fungsi berhenti untuk setiap bilangan bulat $n \geq 0$.

Tugas mandiri M2

Yang dikumpulkan

- 1 Buat empat bentuk fungsi sesuai materi hari ini.
- 2 Buat fungsi `polinom(x, koef)` untuk mengevaluasi polinom.
- 3 Buat fungsi `stat_ringkas` yang mengembalikan beberapa nilai dan gunakan unpacking.
- 4 Buat contoh scope lokal dan global, lalu jelaskan hasilnya.
- 5 Buat `proses(x, f)` dan `pipeline(x, ops)`.
- 6 Buat satu fungsi rekursif untuk barisan sederhana.

Format

Boleh menggunakan Google Colab, Jupyter, Spyder, VS Code, atau IDE Python lain. Kode harus dapat dijalankan ulang dan diberi komentar singkat.

Hal yang perlu dikuasai

- Membuat fungsi dengan input dan output jelas.
- Menggunakan return secara tepat.
- Menelusuri scope.
- Mengirim fungsi sebagai argumen.
- Menulis rekursi dengan base case.

Arah M3

- Representasi algoritma.
- List, tuple, dictionary.
- Pseudocode.
- Array NumPy dan comprehension.

Fungsi yang baik membuat program lebih dekat dengan cara berpikir matematis: jelas inputnya, jelas transformasinya, jelas outputnya.